CS 6210: Matrix Computations

Sparse direct solvers

David Bindel

2025-09-22

Right and left

In this lecture, we will consider various types of sparse direct factorization methods. We'll focus throughout on Cholesky, simply to avoid the awkwardness associated with pivoting. Before discussing sparse Cholesky factorizations, though, it's worth re-considering the dense case. Throughout this lecture we will use the upper triangular form of the factorization (i.e. $A = R^T R$ where $R = L^T$).

So far, we have mostly considered *right-looking* (aka *downward-looking*) factorization method. The simplest version of these methods can be seen as follows:

1. Partition $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & A_{22} \end{bmatrix}$
2. Compute $r_{11} = \sqrt{a_{11}}$ and $r_{12} = a_{12}/r_{11}$
3. Recursively factor $S = A_{22} - r_{12}^T r_{12}$

In code, this looks like:

```
function mychol_rl!(A)
    n = size(A)[1]
    for k = 1:n

        # Compute row of R
        A[k,k] = sqrt(A[k,k])
        A[k,k+1:n] ./= A[k,k]

        # Schur complement update (just upper triangle)
        for i = k+1:n
            A[i,k+1:n] .-= A[k,i]*A[k,k+1:n]
        end
```

```
    end
    UpperTriangular(A)
end
```

In contrast, *left-looking* (aka *upward-looking*) factorization methods recurse in the other direction

1. Partition $A = \begin{bmatrix} A_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$
2. Factor $A_{11} = R_{11}^T R_{11}$ recursively
3. Compute the last column by $r_{12} = R_{11}^{-T} a_{12}$ and $r_{22} = \sqrt{a_{22} - \|r_{12}\|^2}$

In code, this looks like:

```
function mychol_ll!(A)
    n = size(A)[1]
    A[1,1] = sqrt(A[1,1])
    for k = 2:n

        # Back substitution with R11^T + Schur compl
        for i=1:k-1
            A[i,k] -= A[1:i-1,i]'*A[1:i-1,k]
            A[i,k] /= A[i,i]
            A[k,k] -= A[i,k]*A[i,k]
        end

        # Compute next diagonal
        A[k,k] = sqrt(A[k,k])
    end
    UpperTriangular(A)
end
```

The two algorithms both do the same operations, and both have $O(n^3)$ complexity. But each has a slightly different interpretation to it, and these different interpretations will prove useful when we consider sparse matrix factorization.

Band and skyline solvers

We begin with the case of band matrices. These have the great advantage of looking somewhat similar to dense matrices (and much less complicated than the general sparse case).

The *bandwidth* b of a matrix is the smallest non-negative integer b such that $a_{ij} = 0$ for $|i - j| > b$. Hence, diagonal matrices have bandwidth 1, bidiagonal and tridiagonal matrices have bandwidth 2, and so forth. In the nonsymmetric case, we may distinguish between the upper and the lower bandwidth.

For symmetric positive definite matrices of bandwidth b, the Cholesky factors also have bandwidth b. To see why, consider one step of Cholesky factorization for a pentadiagonal matrix (bandwidth $b = 2$):

$$A = \begin{bmatrix} \times & \times & \times & & & & & & \\ \times & \times & \times & \times & & & & & \\ \times & \times & \times & \times & \times & & & & \\ & \times & \times & \times & \times & \times & & & \\ & & \times & \times & \times & \times & \times & & \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \\ & & & & \times & \times & \times & \times & \times \\ & & & & & \times & \times & \times & \times \\ & & & & & & \times & \times & \times \end{bmatrix}$$

The first step of Cholesky factorization takes the square root of the $(1, 1)$ element, scales the first row, and (conceptually) zeros out the subdiagonal entries of the first column. If we mark the modified entries with stars, we have

$$\begin{bmatrix} * & * & * & & & & & & \\ 0 & \times & \times & \times & & & & & \\ 0 & \times & \times & \times & \times & & & & \\ & \times & \times & \times & \times & \times & & & \\ & & \times & \times & \times & \times & \times & & \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \\ & & & & \times & \times & \times & \times & \times \\ & & & & & \times & \times & \times & \times \\ & & & & & & \times & \times & \times \end{bmatrix}.$$

The entries multiplied in the Schur complement update are those in the $(2 : b) \times (2 : b)$ block

$$\begin{bmatrix} * & * & * & & & & & & \\ 0 & * & * & \times & & & & & \\ 0 & * & * & \times & \times & & & & \\ & \times & \times & \times & \times & \times & & & \\ & & \times & \times & \times & \times & \times & & \\ & & & \ddots & \ddots & \ddots & \ddots & \ddots & \\ & & & & \times & \times & \times & \times & \times \\ & & & & & \times & \times & \times & \times \\ & & & & & & \times & \times & \times \end{bmatrix}.$$

We now observe that the Schur complement continues to have bandwidth b, and so this same pattern will repeat throughout the factorization.

Because both the matrix A and the Cholesky factor have the same structure, we can write a version of Cholesky that transforms a packed version of the (upper triangle) of the matrix A to a packed version of its Cholesky factor. The packed storage format we use has $b+1$ rows (for the main diagonal and each of the b superdiagonals) and n columns:

$$B = \begin{bmatrix} \cdot & \cdot & a_{13} & \dots & a_{n-2,n} \\ \cdot & a_{12} & a_{23} & \dots & a_{n-1,n} \\ a_{11} & a_{22} & a_{33} & \dots & a_{n,n} \end{bmatrix}$$

The indexing convention is $b_{dj} = a_{j-m+d,j}$, $m = 1+\beta$, where β is the bandwidth. Our right-looking band Cholesky is almost identical to the right-looking Cholesky from before, except that we introduce an indexing function that maps our ordinary (i,j) entries to the (d,j) indexing used in the packed format.

```
function bandchol_rl!(B)
    m, n = size(B)
    f(i,j) = (i-j+m)+(j-1)*m
    for k = 1:n
        # -- Compute row of R --
        B[f(k,k)] = sqrt(B[f(k,k)])
        for j = k+1:min(k+m-1,n)  B[f(k,j)] /= B[f(k,k)]  end

        # -- Update Schur complement --
        for j = k+1:min(k+m-1,n)
            for i = k+1:j
                B[f(i,j)] -= B[f(k,i)]*B[f(k,j)]
            end
        end
    end
    B
end
```

In thinking about the right-looking algorithm, our emphasis is on the Schur complements and their structure. In constrast, the left-looking version of the algorithm focuses on linear solves. Suppose that

$$A = \begin{bmatrix} A_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

and $A_{11} = R_{11}^T R_{11}$. We know that $r_{12} = R_{11}^{-T} a_{12}$; and if a_{12} is zero up to the kth component, then the same is true of r_{12} because of the way the forward substitution algorithm works. In

our pentadiagonal example above, we have (for example)

$$\begin{bmatrix} \times & & & \\ \times & \times & & \\ \times & \times & \times & \\ & \times & \times & \times \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \times \\ \times \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \times \\ \times \end{bmatrix}$$

Therefore, the last column of R has the same structure as the last column of A; and recursing backward, we see this is true for the other columns as well.

The left-looking version of the Cholesky algorithm in band form is below:

```
function bandchol_ll!(B)
    m, n = size(B)
    B[m,1] = sqrt(B[m,1])
    for k = 2:n
        d1 = max(1,m-k+1)
        for d = d1:m-1

            # Compute a step of forward substitution
            B[d,k] -= B[(d1-d)+m:m-1,k-m+d]' * B[d1:d-1,k]
            B[d,k] /= B[m,k-m+d]

            # Subtract off from the last diagonal element
            B[m,k] -= B[d,k]*B[d,k]

        end

        # Finish with square root
        B[m,k] = sqrt(B[m,k])
    end
    B
end
```

As with the right-looking version of the algorithm, the left-looking band Cholesky can be derived from the dense version by keeping the exact same logic, just with different indexing of the relevant data structure.

Nonsymmetric band solvers may involve pivoting, but even then the band structure cannot increase by very much. In each case, by using compact representations of band matrices, we can compute an LU or Cholesky factorization using $O(nb^2)$ time and $O(nb)$ space. And, once the factorization is computed, forward and backward substitution steps then take $O(nb)$ time as well.

A generalization of band solvers is the *profile* or *skyline* solver. A skyline matrix format has a different bandwidth for each column of the upper triangular part (row of the upper triangular part); for example, consider the matrix

$$\begin{bmatrix} \times & \times & \times & & & & & & \times \\ \times & \times & \times & & & & & & \times \\ \times & \times & \times & \times & \times & & & & \times \\ & & \times & \times & \times & \times & & & \times \\ & & \times & \times & \times & \times & \times & & \times \\ & & & \times & \times & \times & \times & \times & \times \\ & & & & \times & \times & \times & \times & \times \\ & & & & & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times & \times & \times & \times \end{bmatrix}$$

which has the bandwidth vector

$$\begin{bmatrix} 0 & 1 & 2 & 1 & 2 & 2 & 2 & 2 & 8 \end{bmatrix}.$$

A natural approach to storing this (similar to the compressed sparse column format) is to just keep a list of the structurally nonzero entries of the upper triangle in column major order, along with an indexing array that indicates the start of each column (with an index in position $n+1$ one past the end of the data for the last column). That is, we have a vector `colptr` with entries

$$\begin{bmatrix} 1 & 2 & 4 & 7 & 9 & 12 & 15 & 18 & 21 & 30 \end{bmatrix}$$

Our analysis of the fill pattern in the left-looking Cholesky algorithm generalizes naturally to the skyline format as well.

General sparse direct methods

Supppse A is positive definite and $A = R^T R$ is the Cholesky factorization. Will R also be sparse? The answer depends in a somewhat complicated way on the structure of the graph associated with the matrix A and the order in which variables are eliminated. Except in very special circumstances, there will generally be more nonzeros in R than there are in A; these extra nonzeros are referred to as *fill*.

The standard approach for minimizing fill is to apply a *fill-reducing ordering* to the variables; that is, use a factorization

$$PAP^T = R^T R$$

where the permutation P is chosen to minimize the number of nonzeros in R. In the nonsymmetric case, one considers

$$PAQ = LU,$$

where Q is a column permutation chosen to approximately minimize the fill in L and U, and P is the row permutation used for stability. One can relax the standard partial pivoting condition, choosing the row permutation P to balance the desire for numerical stability against the desire to minimize fill.

The problem of finding an elimination order that minimizes fill is NP-hard, so it is hard to say that any ordering strategy is really optimal. But there is canned software for some heuristic orderings that tend to work well in practice. From a practical perspective, then, the important thing is to remember that a fill-reducing elimination order tends to be critical to using sparse Gaussian elimination in practice.

Tree elimination

Consider the following illustrative example of how factoring a sparse matrix can lead to more or less dense factors depending on the order of elimination. Putting in $\times$ to indicate a nonzero element, we have

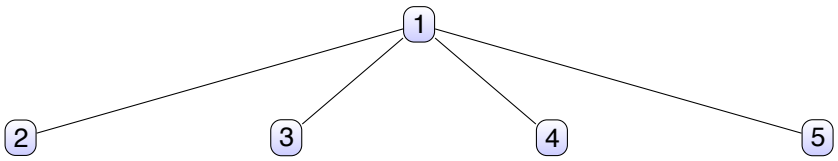
$$\begin{bmatrix} \times & \times & \times & \times & \times \\ \times & \times & & & \\ \times & & \times & & \\ \times & & & \times & \\ \times & & & & \times \end{bmatrix} = \begin{bmatrix} \times & & & & \\ \times & \times & & & \\ \times & \times & \times & & \\ \times & \times & \times & \times & \\ \times & \times & \times & \times & \times \end{bmatrix} \begin{bmatrix} \times & \times & \times & \times & \times \\ & \times & \times & \times & \times \\ & & \times & \times & \times \\ & & & \times & \times \\ & & & & \times \end{bmatrix}.$$

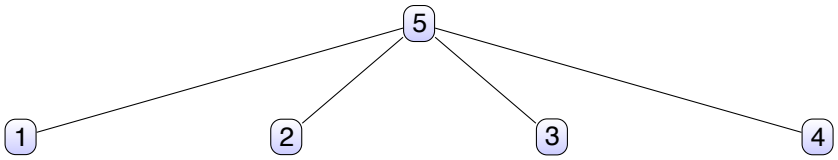
That is, R has many more nonzeros than A. These nonzero locations that appear in R and not in A are called *fill-in*. On the other hand, if we cyclically permute the rows and columns of A, we have

$$\begin{bmatrix} \times & & & & \times \\ & \times & & & \times \\ & & \times & & \times \\ & & & \times & \times \\ \times & \times & \times & \times & \times \end{bmatrix} = \begin{bmatrix} \times & & & & \\ & \times & & & \\ & & \times & & \\ & & & \times & \\ \times & \times & \times & \times & \times \end{bmatrix} \begin{bmatrix} \times & & & & \times \\ & \times & & & \times \\ & & \times & & \times \\ & & & \times & \times \\ & & & & \times \end{bmatrix}.$$

That is, the factorization of PAP^T has *no* fill-in.

A sparse matrix A can be viewed as an *adjacency matrices* for an associated graphs: make one node for each row, and connect node i to node j if $A_{ij} \neq 0$. The graphs for the two "arrow" matrices above are:





These graphs of both our example matrices are *trees*, and they differ only in how the nodes are labeled. In the original matrix, the root node is assigned the first label (pre-ordered labeling); in the second matrix, the root node is labeled after all the children (post-ordered labeling). Clearly, the latter label order is superior for Gaussian elimination. This turns out to be a general fact: if the graph for a (structurally symmetric) sparse matrix S is a tree, and if the labels are ordered so that each node appears after any children it may have, then there is no fill-in: that is, R has nonzeros only where S has nonzeros.

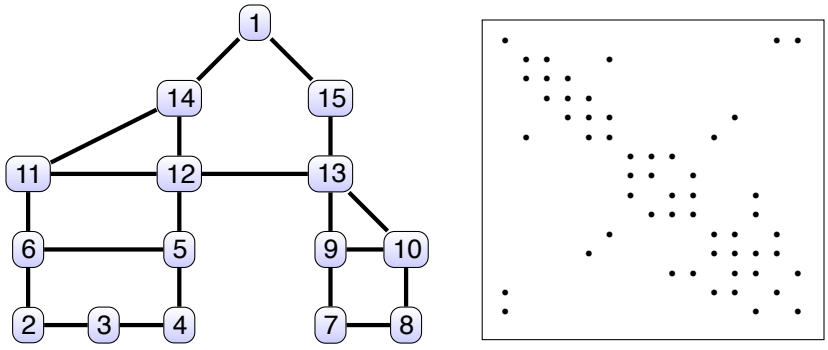
Why should we have no fill when factoring a matrix for a tree ordered from the leaves up? To answer this, we think about what happens in the first step of Gaussian elimination. Our original matrix has the form

$$S = \begin{bmatrix} \alpha & v^T \\ v & S_{22} \end{bmatrix}$$

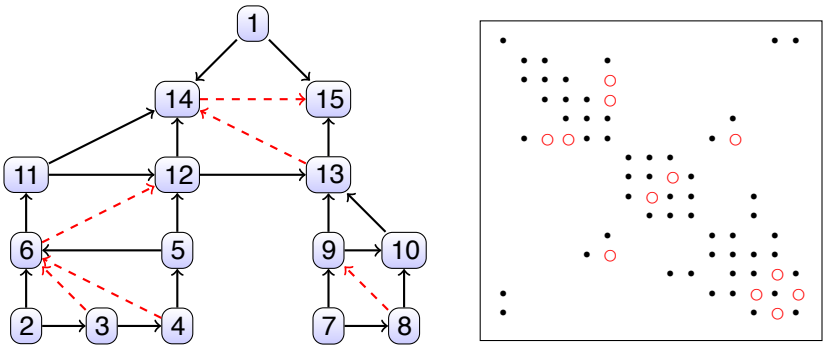
The first row of R is identical to the first row of S, and the first column of L has the same nonzero structure as the first column of A, so we are fine there. The only question is about the nonzero structure of the Schur complement $S_{22} - vv^T/\alpha$. Note that the update vv^T/α has nonzeros only where v_i and v_j are both nonzero — that is, only when nodes i and j are both connected to node 1. But node 1 is a leaf node; the only thing it connects to is its parent! So if p is the index of the parent of node 1 in the tree, then we only change the (p, p) entry of the trailing submatrix during the update — and we assume that entry is already nonzero. Thus, the graph associated with the Schur complement is the same as the graph of the original matrix, but with one leaf trimmed off.

A more interesting example

Now let us turn to a more interestingly sparse example for which (unlike our examples so far) there is no ordering that completely does away with fill.



If we plot the graph of the Cholesky factor along with the sparsity structure of $R^T + R$, we might see that there is some structure to the fill (indicated with dashed red lines in the graph and with red open circles in the "spy ploy" of $R + R^T$).



The question, then, is how can we analyze the sparsity for examples like this?

From the "right-looking" perspective, we have a sequence of graphs associated with successive Schur complements in the elimination process. When we eliminate node k, we get the next graph by removing node k and connecting all its neighbors together to form a clique. This is the perspective we took above in our discussion of tree graphs.

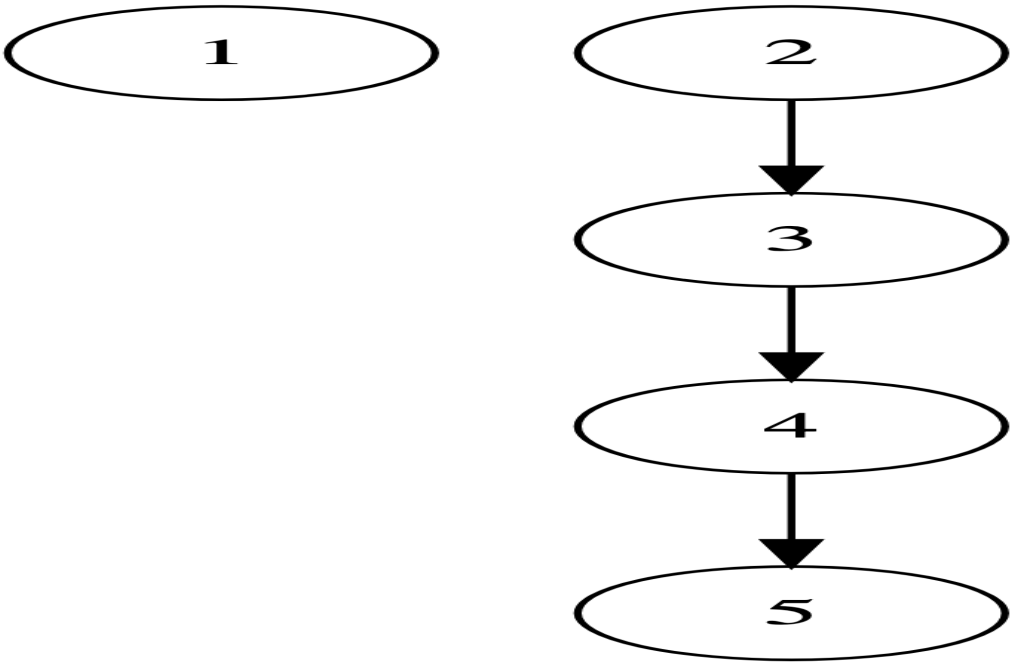
From the "left-looking" perspective, we have a sequence of directed graphs associated leading submatrices $\mathcal{G}_k$ for the Cholesky factor. As a concrete case, let's consider the step of advancing from $\mathcal{G}_5$ to $\mathcal{G}_6$ in the example above. This involves solving the linear system

$$\begin{bmatrix} r_{11} & & & & \\ & r_{22} & & & \\ & r_{23} & r_{33} & & \\ & & r_{34} & r_{44} & \\ & & & r_{45} & r_{55} \end{bmatrix} \begin{bmatrix} r_{16} \\ r_{26} \\ r_{36} \\ r_{46} \\ r_{56} \end{bmatrix} = \begin{bmatrix} 0 \\ a_{26} \\ 0 \\ 0 \\ a_{56} \end{bmatrix}$$

Forward substitution gives us

$$\begin{aligned} r_{16} &= 0 \\ r_{26} &= a_{26}/r_{22} \neq 0 \\ r_{36} &= (0 - r_{23}r_{26})/r_{33} \neq 0 \\ r_{46} &= (0 - r_{34}r_{36})/r_{44} \neq 0 \\ r_{56} &= (a_{56} - r_{45}r_{46})/r_{55} \neq 0 \end{aligned}$$

The flow of information through forward substitution can be summarized in the graph $\mathcal{G}_5$:



The fact that a_{26} is nonzero implies that r_{26} is nonzero, which in turn implies the same of everything reachable from node 2: that is, r_{36}, r_{46}, and r_{56} are all nonzero.

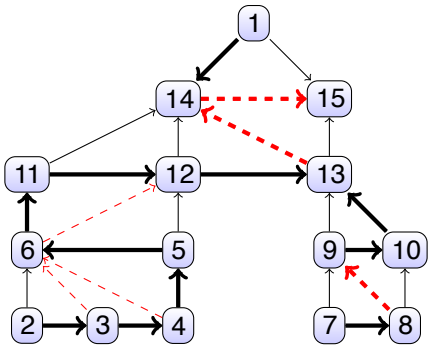
Let's consider more generally the process of computing column $k+1$ of R given columns 1 through k. The forward substitution recurrence is

$$r_{j,k+1} = \left(a_{j,k+1} - \sum_{i<j} r_{ij} r_{i,k+1} \right) / r_{jj}.$$

In terms of the nonzero structure of R:

- If $a_{j,k+1} \neq 0$, then $r_{j,k+1} \neq 0$. That is, if j connects to $k+1$ in the graph of A, then there is an edge from j to $k+1$ in the graph of R.
- If $r_{i,k+1} \neq 0$ and $r_{ij} \neq 0$, then $r_{j,k+1} \neq 0$.
- More generally, $r_{j,k+1} \neq 0$ whenever $a_{i,k+1} \neq 0$ and there is a directed path from i to j in $\mathcal{G}_k$ (in which case we say j is *reachable* from i).

Establishing that j is reachable from i only requires that we show one path. In general, though, there may be many redundant paths. A depth-first or breadth-first search from i defines a *spanning tree* for $\mathcal{G}_k$ that can be used to answer reachability queries without such redundancy. Depth-first search is particularly nice here because it has nesting structure: in the depth-first search tree for $\mathcal{G}_n$ are the depth-first search trees for all $\mathcal{G}_k$ for $k < n$. The tree for $\mathcal{G}_n$ is called the *elimination tree* for A. The elimination tree for our example graph is shown with bold edges below:



The elimination tree can be computed in almost $\text{nnz}(A)$ time with the use of union-find data structures. Once we have the elimination tree, it can be used for several tasks. As we've hinted, the elimination tree can be used to compute the nonzero structure of R; it can also be used identify supernodes (columns with fill patterns that are a good target for level 3 BLAS operations), and to identify opportunities for parallelism (since disjoint subtrees of the elimination tree can be eliminated independently).

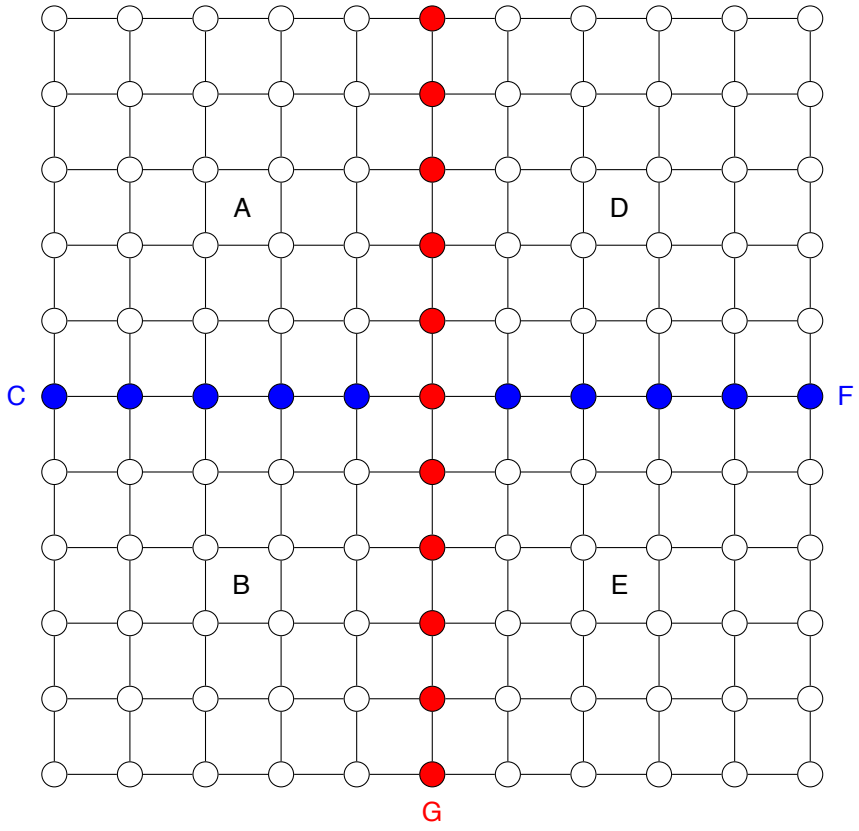
The analysis of the nonzero structure of R via graph methods is referred to as *symbolic factorization*, and typically precedes the *numerical factorization* phase in which the R is actually formed.

For many more details, we refer to *Direct Methods for Sparse Linear Systems* by Tim Davis.

Nested dissection

Tree-structured matrices are marvelous because we can do everything in $O(n)$ time: we process the tree from the leaves to the root in order to compute L and U, then recurse from the root to the leaves in order to do back substitution with U, and then go back from the leaves to the root in order to do forward substitution with L. Sadly, many of the graphs we encounter in practice do not look like trees. However, we can often profitably think of clustering nodes so that we get a *block* structure associated with a tree.

For illustrative purposes, let us consider Gaussian elimination on a matrix whose graph is a regular $n \times n$ mesh. Such a matrix might arise, for example, if we were solving Poisson's equation using a standard five-point stencil to discretize the Laplacian operator. We then think of cutting the mesh in half by removing a set of separator nodes, cutting the halves in half, and so forth. This yields a block structure of a tree consisting of a root (the separator nodes) and two children (the blocks on either side of the separator). We can now dissect each of the sub-blocks with a smaller separator, and continue on in this fashion until we have cut the mesh into blocks containing only a few nodes each. Figure 1 illustrates the first two steps in this process of *nested dissection*.



$$S = \begin{bmatrix} S_{AA} & & S_{AC} & & & & S_{AG} \\ & S_{BB} & S_{BC} & & & & S_{BG} \\ S_{CA} & S_{CB} & S_{CC} & & & & S_{CG} \\ & & & S_{DD} & & S_{DF} & S_{DG} \\ & & & & S_{EE} & S_{EF} & S_{EG} \\ & & & S_{FD} & S_{FE} & S_{FF} & S_{FG} \\ S_{GA} & S_{GB} & S_{GC} & S_{GD} & S_{GE} & S_{GF} & S_{GG} \end{bmatrix}$$

Figure 1: Nested dissection on a square mesh. We first cut the graph in half with the red separator G, then further disect the halves with the blue separators C and F. Nodes in A, B, D, and E are only connected through these separator nodes, which is reflected in the sparsity pattern of the adjacency matrix S when it is ordered so that the separators appear after the things they separate.

We can get a lower bound on the cost of the factorization by figuring out the cost of factoring the Schur complement associated with G, C, F, etc. After we eliminate everything except the nodes associated with G, we pay about $2n^3/3$ flops to factor the remaining (dense) n-by-n Schur complement matrix G. Similarly, we pay about $2(n/2)^3/3$ time to factor the dense $(n/2)$-by-$(n/2)$ complements associated with the separators C and F. Eliminating all four separators then costs a total of $\approx 10n^3/12$ flops. Now, think of applying nested dissection to blocks A, B, D, and E; eliminating the Shur complements associated with separators inside each of these blocks will take about $5(n/2)^3/6$ flops; all four together cost a total of $4(5(n/2)^3/6) = (1/2)(5n^3/6)$ flops to factor. If we keep recursing, we find that the cost of factoring Schur complements associated with all the separators looks like

$$\frac{5}{6}n^3\left(1+\frac{1}{2}+\frac{1}{4}+...\right) \approx \frac{5}{3}n^3.$$

It turns out that forming each Schur complement is asymptotically not more expensive than eliminating it, so that the overall cost of doing nested dissection on an $n \times n$ mesh with $N = n^2$ unknown is also $O(n^3) = O(N^{1.5})$. It also turns out that the fill-in is $O(N \log N)$[1].

Now think about doing the same thing with a three-dimensional mesh. In this case, the top-level separators for an $n \times n \times n$ mesh with $N = n^3$ unknowns would involve n^2 unknowns, and we would take $O(n^6) = O(N^2)$ time to do the elimination, and $O(N^{4/3})$ fill. This relatively poor scaling explains why sparse direct methods are attractive for solving 2D PDEs, but are less popular for 3D problems.

Sparse solvers in practice

Well-tuned sparse elimination codes do not have quite the flop rate of dense linear algebra, but they are nonetheless often extremely fast. In order to get this speed, though, quite a bit of engineering is needed. In the remainder of these notes, we sketch some of these engineering aspects – but we do so largely to convince you that you are better off using someone else's sparse solver code than rolling your own! If you want all the gory details, I again highly recommend the book *Direct Methods for Sparse Linear Systems*.

Symbolic factorization

Typical sparse Cholesky codes involve two stages: a *symbolic factorization* stage in which the nonzero structure of the factors is computed, and a *numerical factorization* stage in which we fill in that nonzero structure with actual numbers. One advantage of this two-stage approach is that we can re-use the symbolic factorization when we are faced with a series of matrices

[1] The explanation of why is not so hard, at least for regular 2D meshes, but it requires more drawing than I feel like at the moment. The paper "Nested Dissection of a Regular Finite Element Mesh" by Alan George (SIAM J. Numer. Anal. 10(2), April 1973) gives a fairly readable explanation for the curious.

that all have the same nonzero structure. This happens frequently in nonlinear PDE solvers, for example: the Jacobian of the discretized problem changes at each solver step (or each time step), but the nonzero structure often remains fixed.

(Approximate) minimum degree ordering

When we have a clear geometry, nested dissection ordering can be very useful. Indeed, nested dissection is useful in some cases even when we have "lost" the geometry – we can use spectral methods (which we will describe later in the course) to find small separators in the graph. But in some cases, there is no obvious geometry, or we don't want to pay the cost of computing a nested dissection ordering. In this case, a frequent alternative approach is a *minimum degree ordering*. The idea of minimum degree ordering is to search the Schur complement graph for the node with smallest degree, since the fill on eliminating that variable is bounded by the square of the degree. Then we eliminate the vertex, update the degrees of the neighbors, and repeat. Unfortunately, this is expensive to implement in the way described here – better variants (using quotient graphs) are more frequently used in practice.

Cache locality

In order to get good use of level 3 BLAS, sparse direct factorization routines often identify dense "supernodal" structure in the factor. We have already seen one case where this happens in our discussion of nested dissection: we get dense blocks arising from separator Schur complements. The main alternative to supernodal solvers is the family of *multifrontal* solvers, which also are able to take advantage of level 3 BLAS.

Elimination trees and parallelism

As discussed earlier, elimination trees are useful not only for identifying the structure of R, but also for finding dense "supernodal" structures and for finding opportunities for parallelism: disjoint subtrees of the elimination tree do not directly interact, and can be eliminated in parallel in the numerical factorization.